

# Assignment 3 - CoW: Copy On Write

April 17, 2025

## 1 Background

Virtual memory provides a level of indirection: the kernel can intercept memory references by marking PTEs invalid or read-only, leading to page faults, and can change what addresses mean by modifying PTEs. There is a saying in computer systems that any systems problem can be solved with a level of indirection.

This assignment explores an example: copy-on-write fork.

**Quote** “All problems in computer science can be solved by another level of indirection.”

## 2 The problem

The `fork()` system call in xv6 copies all of the parent process’s user-space memory into the child. If the parent is large, copying can take a long time. Worse, the work is often largely wasted: `fork()` is commonly followed by `exec()` in the child, which discards the copied memory, usually without using most of it. On the other hand, if both parent and child use a copied page, and one or both writes it, the copy is truly needed.

## 3 The solution

Your goal in implementing copy-on-write (COW) `fork()` is to defer allocating and copying physical memory pages until the copies are actually needed, if ever. COW `fork()` creates just a pagetable for the child, with PTEs for user memory pointing to the parent’s physical pages. COW `fork()` marks all the user PTEs in both parent and child as read-only. When either process tries to write one of these COW pages, the CPU will force a page fault. The kernel page-fault handler detects this case, allocates a page of physical memory for the faulting process, copies the original page into the new page, and modifies the relevant PTE in the faulting process to refer to the new page, this time with the PTE marked writeable. When the page fault handler returns, the user process will be able to write its copy of the page.

---

Build commit hash:

COW `fork()` makes freeing of the physical pages that implement user memory a little trickier. A given physical page may be referred to by multiple processes' page tables, and should be freed only when the last reference disappears. This mechanism is called **reference counting**.

### 3.1 Overview Guidance

- 1 Modify `mm_copy()` to map the parent's physical pages into the child (see `mm_mappages_cow()`), instead of allocating new pages. Clear `PTE_W` in the PTEs of both child and parent for pages that have `PTE_W` set.
- 2 Modify `handle_pgfault()` in *os/trap.c* to recognize page faults from usermode. When a write page-fault occurs on a COW page that was originally writeable, allocate a new page with `kallocpage()`, copy the old page to the new page, and install the new page in the PTE with `PTE_W` set.
- 3 Ensure that each physical page is freed when the last PTE reference to it goes away – but not before.
- 4 A good way to do this is to keep, for each physical page, a "reference count" of the number of user page tables that refer to that page. Set a page's reference count to one when `mm_mappages()` allocates it from `kallocpage()`. Increment a page's reference count when `fork(mm_copy, mm_mappages_cow)` causes a child to share the page, and decrement a page's count each time any process drops the page (`freevma()`) from its page table. `kfreepage()` should only be called if its reference count is zero, i.e., no more user processes are using it.
- 5 It may be useful to have a way to record, for each PTE, whether it is a COW mapping. You can use the RSW (reserved for software) bits in the RISC-V PTE for this.
- 6 If a COW page fault occurs and there's no free memory, the process should be killed.
- 7 Pages that were originally read-only (not mapped `PTE_W`, like pages in the text segment) should remain read-only and shared between parent and child; a process that tries to write such a page should be killed.
- 8 Modify `copy_to_user` to use the same scheme as page faults when it encounters a COW page.

**Reference Counting** Reference counting is a shared resource management technique used to track and manage the **lifecycle** of shared objects. The core idea is that whenever an object is referenced, its reference count increases; whenever a reference no longer uses the object, the reference count decreases. When an object's reference count drops to zero, it indicates that no part of the system is referencing the object anymore, at which point the system can safely release the resources occupied by the object.

## 3.2 Implementation Hints

1 Search the code repository for Assignment 3 to examine the hints in the code.

2 Reference counting applies solely to user pages (not page tables). The supporting code is in *vm.c*.

If you find references suggesting that reference counting can be implemented in either *kalloc.c* or *vm.c*, please note that for this assignment, **you must not implement reference counting in the kalloc module**". Doing so may cause our ktest module to fail.

3 How xv6 reclaims user pages:

- `freeproc` calls `mm_free`.
- `mm_free` invokes `mm_free_vmas` which iterates through the vma list and releases each vma via `freevma`.
- `freevma` locates the physical pages mapped by its virtual memory region and releases them.
- Then `mm_free` calls `freepgt` to release physical pages used by the page tables themselves.
- Finally, `mm_free` releases the `struct mm` memory.

4 xv6 allocates physical pages for user processes in the functions `mm_mappages` and `mm_remap`.

5 We've simplified `mm_remap` to only handle VMA expansion. User programs calls `sbrk` system calls to adjust brk size(heap region), system calls then modifies the region marked by `p->vma_brk` and uses `mm_remap` to remap this VMA.

6 You may have learned in C/C++ courses that when using `malloc` and `free` for dynamic memory allocation, you should beware of two critical scenarios: Use-after-free and Double Free. The former occurs when you continue using an object after it has been freed, while the latter happens when you free the same object twice, which can corrupt the memory allocator.

7 In the kernel, we utilize the **kalloc** module for dynamic allocation of physical pages. User pages represent the "dynamically allocated" objects in this context.

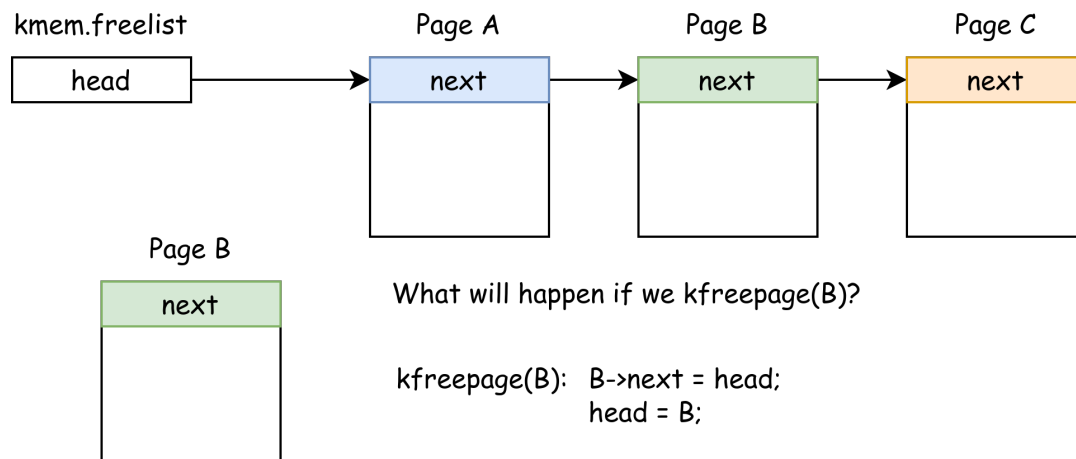
8 When reference counting is neglected, you will likely encounter Use-after-free, Double Free, and memory leaks. The first two scenarios often cause the kernel to enter highly unstable states. Consider the following cases:

- When sharing a physical page, the reference count was not incremented. A physical page was shared by two processes, but the reference count was only 1. When one of the processes exited, the reference count was decremented to 0, and the physical page was released. At this point, the other process was still using the physical page.
- The page was released without considering the reference count. This caused a shared page to be freed twice, which corrupts the **kalloc** module.

9 Detailed Explanation of `kallocpage/kfreepage`: We use a singly linked list to connect all free physical pages. A `struct linklist` is placed at the beginning of each page, and the kernel maintains the head pointer of the list, `kmem.freelist`.

Since only free pages have a `next` pointer, for allocated pages, users neither need to nor should care about the fact that there was once a `next` pointer in the page header.

So, let's think about what happens in the case of a Double-free: Suppose A, B, and C are three allocated pages, and consider the following sequence of `kfreepage` calls: C, B, A, B. The last `kfreepage(B)` is a Double Free.



Next, consider what happens if we call `kallocpage` three times after a double-free scenario. We encourage you to address this question in your report.

### 3.3 Debugging Hints

- 1 Tracking `kallocpage`/`kfreepage`: You can add `#define LOG_LEVEL_DEBUG` at the **first line** of `kalloc.c` (before any `#include` directives), and the kernel will print debug information when these two functions are called.
- 2 Locating the caller in logs: In the debug logs, we can find the caller of the function: *alloc/free PA, by caller*. The *caller* is an address like `0xffffffff8020xxxx`, which corresponds to the value of the `ra` register when these functions are called. This address points to a kernel code location. You can find its corresponding assembly in `build/kernel.asm`, or use the `addr2line` tool to quickly locate the C source line number.

```

1 [DEBUG 0,2] kfreepage: free: 0x0000000080d14000, called by 0xffffffff80208b20
2 [DEBUG 0,2] kfreepage: free: 0x0000000080d13000, called by 0xffffffff8020954c
3 sh >> QEMU: Terminated
4
5 /D/s/a/src $ addr2line -e build/kernel
6 0xffffffff8020954c      <-- Paste and Enter.
7 ~/a3-cow/src/os/vm.c:228
8 0xffffffff80208b20
9 ~/a3-cow/src/os/vm.c:209 (discriminator 1)
10
```

- 3 These issues are particularly troublesome—they cause memory corruption and remain hidden at the initial point of failure, only manifesting later in unexpected behavior (second-site crashes). So, how do we debug them?

**Tracking Memory Corruption:** When allocating a page from the `kalloc` module, we fill the newly returned page with a specific garbage byte (0xaf). Similarly, when freeing a page, we fill it with another garbage byte (0xdd).

This way, when a second-site crash occurs (often a Kernel Panic), we may notice that memory or registers contain these garbage bytes (e.g., during a PageFault, where a pointer stored in the page was overwritten by garbage). This helps identify issues like Use-before-initialization, Use-after-free, or Double-free.

**Recommended Reading:** In Linux + glibc, what tools can we use to track such memory errors, and how do they work? Are there similar tools in the Linux Kernel? You are encouraged to explore and answer this question in your report.

- 4 You can also modify the `kfreepage` function in `kalloc.c` to verify that a page has not already been freed before releasing it. This is a form of defensive programming.
- 5 You can use the `vm_print` function (implemented in `debug.c`, requiring `debug.h`) to print a page table. In user mode, you can use `ktest(KTEST_PRINT_USERPGT, 0, 0)`; to delegate `ktest` to print the user page table.

### 3.4 Startup

Before you start modifying the code, we recommend carefully considering the following questions and encourage you to address them in your report:

- 1 In `xv6`'s kernel memory management module (`kalloc.c`), which function is responsible for allocating physical pages, and which function is responsible for releasing physical pages?
- 2 In `xv6`'s user memory management module (`vm.c`), which functions are responsible for setting up user pages, which functions release user pages, and which functions manipulate user page tables?
- 3 In `mm_copy`, we use `mm_mappages` to map the child process's pages and copy the page contents from the parent process. We observe that this copying might be redundant because calling `exec` after `fork` discards the pages copied during `fork`. So, how do we avoid this unnecessary copying?

Starting from `mm_copy`, modify `mm_mappages` to `mm_mappages_cow`, and implement `mm_mappages_cow` to map the parent process's physical pages into the child process using a Copy-on-Write (CoW) approach.

## 4 Checkpoints

**This assignment is worth 4 points, consisting of 4 Checkpoints and a report.** Each Checkpoint is worth 1 point, and the report does not contribute to the score but is mandatory. Your report should meet the following requirements:

- A report in PDF format.
- Screenshots demonstrating successful completion of each checkpoint.
- For the thought-provoking questions in the guidelines, you are not required to answer them in the report, but we encourage you to think about them and provide your responses.
- Record the total time spent completing this assignment.

- (Optional) Describe any difficulties you encountered while working on this assignment, or suggest additional guidance you think would be helpful.
- (Optional) You may share your approach or thought process in completing the assignment.
- The more concise, the better.

**Requirements** You should use `make run` to start xv6 in single-core mode. For this assignment, we provide `usertest`. After booting the kernel, you should see an entry labeled `cowtest` under the `applist:` prompt. You need to pass `cowtest 1-4`. Consistently passing these tests will be considered as completing the Checkpoint.

Note: The regression test `proctest` may report errors, which is expected. The grading for this assignment is not affected by it.

**Checkpoint Descriptions** Checkpoint 1 tests `fork` when the system has insufficient physical memory. CoW avoids allocating new physical pages by sharing the parent process's physical pages. Checkpoint 2 verifies whether physical pages are leaked. Checkpoint 3 writes to a shared page, triggering Copy-on-Write, and checks whether the child and parent processes have independent physical pages. Checkpoint 4 examines whether the `copy_to_user` function triggers Copy-on-Write.

## 5 Submission

Save your report as `report.pdf` in the same folder, then run `make handin` to create `handin.zip`. Upload the ZIP file to Blackboard for submission.